

Schedulers & GPU Allocation in Multi-User Environments

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India



What you will be able to do after this session

- Explain why ad-hoc GPU sharing collapses beyond a handful of users.
- Compare Slurm and Kubernetes as GPU schedulers and say when each fits.
- Distinguish the three fractional-GPU mechanisms: time-slicing, MPS, MIG.
- Inspect and reason about MIG partitions with `nvidia-smi` — hands-on.
- Measure GPU utilization and spot the classic waste patterns — hands-on.

Eight GPUs, thirty people: the multi-user problem

Without a scheduler

SSH + a spreadsheet + 'is anyone using GPU 3?' on chat. Collisions, OOM-killing each other's jobs, idle GPUs at night, and zero accounting.

With a scheduler

Jobs declare needs (GPUs, memory, time). The scheduler queues, places, isolates, preempts, and accounts. Utilization becomes a managed number.

A scheduler converts hardware from territory into a metered utility.

The two families: Slurm and Kubernetes

Slurm (HPC lineage)

Batch jobs with start and end. Queues/partitions, priorities, fair-share, gang scheduling for multi-node training. The default in research clusters and supercomputers.

Kubernetes (cloud lineage)

Long-running services + jobs. Declarative desired state, self-healing, rich ecosystem (operators, autoscaling). The default for serving and cloud-native ML platforms.

Many orgs run both: Slurm for training farms, Kubernetes for serving — or Kubernetes with batch add-ons doing double duty.

How Slurm hands out GPUs

```
# Ask for resources; the scheduler finds them
$ srun --gres=gpu:2 --mem=64G --time=02:00:00 nvidia-smi

# Batch script header
#SBATCH --gres=gpu:b200:4           # 4 B200s on one node
#SBATCH --partition=train          # which queue
#SBATCH --time=08:00:00            # hard wall-clock limit

$ squeue -u $USER                  # where is my job?
$ sacct -j 12345 --format=Elapsed,AllocGRES,State  # accounting
```

- Slurm sets `CUDA_VISIBLE_DEVICES` so each job sees only its granted GPUs.
- Time limits + fair-share priority are what keep one team from squatting the cluster.

Kubernetes & GPUs — a preview of Week 3

```
# pod.yaml (fragment)
resources:
  limits:
    nvidia.com/gpu: 1      # whole GPUs, surfaced by the device plugin
    memory: "32Gi"
    cpu: "8"
```

- The NVIDIA device plugin / GPU Operator advertises GPUs as schedulable resources.
- The scheduler places pods only on nodes with free GPUs; the runtime wires them in (Session 1's toolkit again).
- GPUs are requested as whole units by default — fractionalization is an add-on (next slide).

How Kubernetes Sees a GPU — The Device Plugin

What the device plugin does

A DaemonSet pod that runs on every GPU node. It queries nvidia-smi, registers each GPU as an allocatable resource (nvidia.com/gpu), and exposes them to the kubelet.

Allocation flow

Scheduler selects node. Kubelet calls device plugin AllocateRPC. Plugin returns mount paths + env vars (CUDA_VISIBLE_DEVICES, device nodes). Container runtime wires them in.

```
# Check what the device plugin has advertised:
kubectl get nodes -o json | \
  jq '.items[].status.capacity |
  select(."nvidia.com/gpu")'
# → {"nvidia.com/gpu": "8"}

# GPU Operator installs driver + plugin + DCGM:
helm install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator --create-namespace

# With MIG: plugin exposes slices as separate
resources
# nvidia.com/mig-2g.45gb: "2"
# nvidia.com/mig-3g.90gb: "1"
kubectl describe node gpu-node-01 | grep nvidia
```

Requests, Limits & QoS — Getting Your Pod Scheduled

requests

The minimum the pod needs.
Scheduler uses this to find a node that can fit the pod. For GPUs: requests must equal limits — no partial GPU by default.

limits

The maximum the pod may use.
Containers exceeding CPU limits are throttled; exceeding memory limits are OOM-killed. GPU limit = GPU request always.

QoS classes

Guaranteed: req=limit (best for GPU pods). Burstable: req < limit.
BestEffort: no req/limit set. GPU pods should always be Guaranteed.

```
# Guaranteed QoS GPU pod — the correct pattern
resources:
```

```
  requests:
```

```
    nvidia.com/gpu: "2"
```

```
    memory: "64Gi"
```

```
    cpu: "16"
```

```
  limits:
```

```
    nvidia.com/gpu: "2"           # must match requests
```

```
    memory: "64Gi"
```

```
    cpu: "16"
```

```
# ⚠ See speaker notes for the warning about limits without requests
```


Fractional GPUs: three sharing mechanisms

Time-slicing

Contexts take turns. No memory isolation — one tenant can OOM another. Fine for bursty notebooks, risky for prod.

MPS

Kernels from several processes run concurrently, sharing SMs. Better utilization; still soft isolation.

MIG

Hardware partitions (e.g. B200 → up to 7 instances), each with dedicated memory and SMs. Real isolation; partition sizes are fixed menus.

Isolation strength: time-slicing < MPS < MIG. Flexibility runs the other way.

CUDA_VISIBLE_DEVICES — How Schedulers Hide GPUs

What it does

An environment variable that tells the CUDA runtime which physical GPU indices are visible. Processes outside the allowed set get no GPU at all — isolation without hardware partitioning.

Scheduler sets it, not you

Slurm injects `CUDA_VISIBLE_DEVICES=2,3` into the job's environment. The job calls `torch.cuda.device(0)` and gets physical GPU 2. Index re-mapping is automatic and invisible to user code.

```
# Slurm injects this before your job starts:  
export CUDA_VISIBLE_DEVICES=2,3
```

```
# Inside your code — sees only 2 devices  
import torch  
print(torch.cuda.device_count()) # → 2  
print(torch.cuda.get_device_name(0)) # →  
physical GPU 2
```

```
# Kubernetes: device plugin does the same  
# via the container's environment block:  
# env:  
#   - name: NVIDIA_VISIBLE_DEVICES  
#     value: "GPU-abc123" ← UUID, not index
```

Time-Slicing — Context Switching at GPU Speed



← GPU time →

Context switches happen at microsecond granularity (GPC scheduler). Not controlled by user code.

When to use

Bursty notebooks. Dev/test instances. Small inference jobs that don't run continuously. Many low-utilisation jobs sharing one GPU.

When NOT to use

Latency-sensitive inference. Multi-process training. Any job that will OOM if a neighbour takes more than its share of memory.

How to enable (K8s)

Device plugin config:

time-slicing:

resources:

- name: nvidia.com/gpu

replicas: 4

Exposes 4× logical GPUs per physical GPU.

Fair-Share Scheduling — Why Your Priority Moves

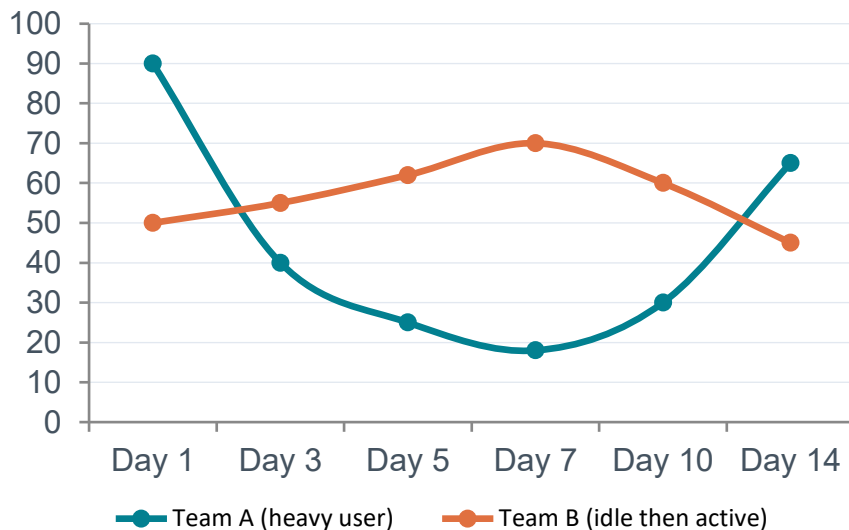
The fair-share formula

Priority = $f(\text{shares}, \text{recent_usage})$. Teams with more allocated shares start higher. The more GPU-hours you've consumed recently, the lower your current priority. Heavy users sink; idle teams float up.

Half-life decay

Usage weight decays exponentially (configurable decay period, typically 7–14 days). A week of idle time restores your priority. Design long jobs to checkpoint — short gaps don't hurt you.

Priority Score Over Time



Preemption — Getting Bumped, Surviving the Bump

1

High-priority job arrives

A job with higher priority (or a reservation) needs the GPUs your job is currently using.

2

Preemption signal

Slurm sends SIGTERM to your job N seconds before SIGKILL (configurable, typically 60–300 s).

3

Checkpoint handler fires

Your code catches the signal, saves model weights + optimizer state + step number to shared storage.

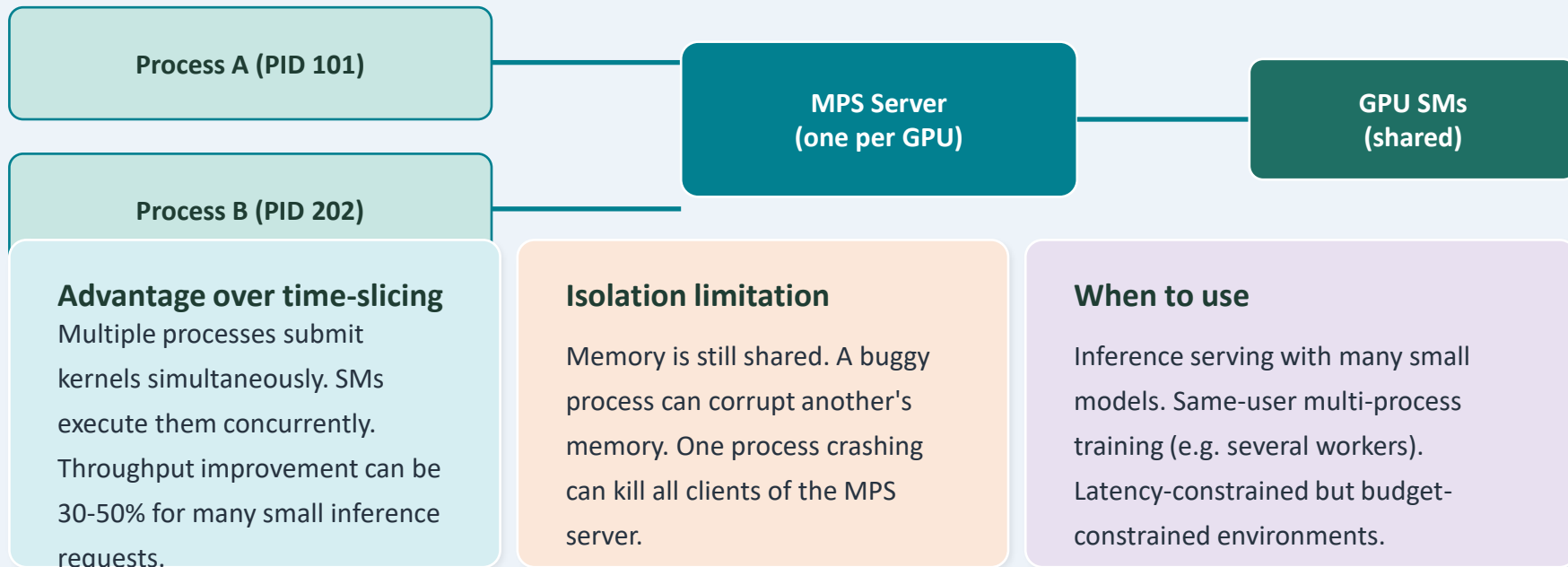
4

Job re-queues

Slurm automatically re-submits the job. On the next allocation it loads the checkpoint and continues from step N.

```
# Catch SIGTERM; save checkpoint before Slurm kills the job
import signal, torch
def checkpoint_handler(signum, frame):
    torch.save({'epoch':epoch,'model':model.state_dict(),'opt':optimizer.state_dict()}, 'ckpt.pt')
    exit(0)
signal.signal(signal.SIGTERM, checkpoint_handler)
# In your training loop: load ckpt.pt if it exists on startup
```

MPS — Multi-Process Service: Concurrent Kernels



Choosing a Sharing Mechanism — Side-by-Side

Attribute	Time-Slicing	MPS	MIG
Memory isolation	None — shared pool	None — shared pool	Hard — per-slice HBM
SM isolation	None — shared	Soft — concurrent	Hard — dedicated SMs
Fault isolation	No — OOM spreads	Partial — crash spreads	Yes — slices independent
Latency impact	High — context switch	Low — concurrent exec	None — dedicated HW
GPU requirement	Any NVIDIA	Volta+ (CC 7.0+)	Ampere A100+ (CC 8.0+)
Max tenants	Unlimited (soft)	48 clients	Up to 7 instances
Best for	Dev notebooks	Inference serving	Multi-team prod clusters

Carving One GPU into Many — Hardware Isolation

What MIG does

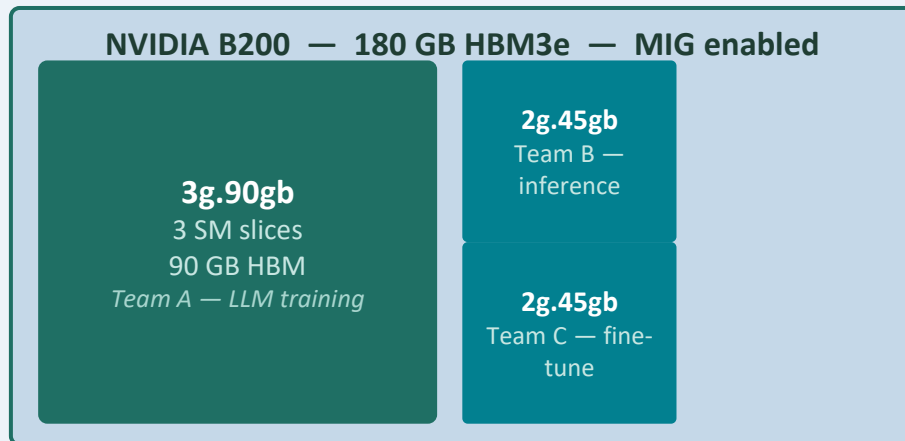
Partitions a single physical GPU into up to 7 isolated instances at the hardware level — dedicated SMs, dedicated HBM, dedicated L2 cache per slice.

Why it matters

One team's runaway job cannot OOM-kill another.
Memory errors in one instance do not corrupt a neighbour. Hard isolation = predictable SLAs.

Data-centre only — why your laptop returns [N/A]

MIG requires Ampere A100 or newer (H100, B200).
Consumer GPUs (GeForce, RTX) share SMs with no hardware memory barriers — partitioning is impossible.



Each slice gets a UUID — containers and schedulers target it like a full GPU

Isolation strength: Time-slicing < MPS < MIG | Scheduling flexibility runs the other way

Takeaway: a MIG slice looks like a smaller, fully isolated GPU to both the OS and the scheduler. The goal is predictable, auditable, multi-tenant GPU sharing — not possible with software-only approaches.

Fairness machinery: queues, priorities, quotas

- Queues/partitions segment hardware by purpose: interactive vs batch vs preemptible.
- Fair-share: heavy recent users sink in priority; idle teams float up — usage self-balances.
- Preemption: low-priority jobs yield to urgent ones; checkpointing makes that survivable.
- Quotas cap concurrent GPUs per team or namespace — the brake on noisy neighbours.
- Backfill: short, well-sized jobs slot into scheduling gaps — declare honest time limits.

GPU Accounting & Chargeback — Making Usage Visible

Why accounting matters

Without accounting, GPU usage is invisible. Teams over-allocate, jobs run overnight with 0% utilisation, and the infrastructure team can't justify buying more hardware.

Slurm accounting

sacct reports per-job: AllocGRES, Elapsed, CPUSTime, MaxRSS.
sreport shows per-user, per-account GPU-hours consumed.
Feed into a cost model: GPU-hours × rate = team bill.

Kubernetes accounting

kube-state-metrics + DCGM Exporter + Prometheus → per-namespace GPU-hour totals. Kubecost or OpenCost add dollar amounts. Chargeback reports go to team leads monthly.

```
# Slurm: GPU-hours used by each user last week
sreport cluster AccountUtilizationByUser \
  Start=$(date -d '7 days ago' +%Y-%m-%d) End=now \
  --tres=gres/gpu -t Hours

# Kubernetes: GPU-hours per namespace (Prometheus query)
# sum by (namespace) (rate(DCGM_FI_DEV_GPU_UTIL[1h]) * on(pod) kube_pod_info) * 720
```

Quotas & Namespace Isolation — The Kubernetes Brake

```
# ResourceQuota caps GPU use per namespace
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a-gpu-quota
  namespace: team-a
spec:
  hard:
    requests.nvidia.com/gpu: "4"
    limits.nvidia.com/gpu: "4"
    requests.memory: "256Gi"

# LimitRange sets per-pod defaults
apiVersion: v1
kind: LimitRange
metadata:
  name: gpu-limit-range
  namespace: team-a
spec:
  limits:
    - type: Container
      default:
        nvidia.com/gpu: "1"
      max:
        nvidia.com/gpu: "2"
```

What ResourceQuota enforces

New pods that would exceed the namespace quota are rejected at admission. Existing running pods are not evicted. The quota is a ceiling, not a guarantee.

Priority classes

PriorityClass lets you mark pods as critical (won't be preempted) vs. low (first to go). Combine with quotas for fine-grained SLA tiers.

Admission webhooks

Custom webhooks can enforce GPU annotations, require team labels, or block oversized requests before they reach the scheduler.

Multi-Node GPU Jobs — Gang Scheduling & Co-Scheduling

The multi-node problem

A 4-node distributed training job needs all 4 nodes simultaneously. If 3 nodes start and 1 is unavailable, the 3 nodes sit idle holding resources. This is the 'stranded GPU' problem.

Gang scheduling (Slurm)

Slurm's default: all tasks of a job step are launched simultaneously. If any node is unavailable the entire job waits in queue — no partial allocations. Simple and safe for HPC clusters.

```
# Slurm: 4 nodes × 8 GPUs each = 32 GPUs, all at
once
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=8
#SBATCH --gres=gpu:8
#SBATCH --time=24:00:00

# Kubernetes: Volcano batch scheduler for gang
scheduling
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
spec:
  minAvailable: 4    # all 4 pods or none
  tasks:
    - replicas: 4
      template:
        spec:
          resources:
            limits:
              nvidia.com/gpu: "8"
```

Topology-Aware Scheduling — NVLink vs PCIe Placement

Why topology matters

On an 8-GPU node, GPUs 0–3 and 4–7 may be in different NVLink domains. All-reduce across GPUs in the same NVLink domain = 900 GB/s (NVLink 5). Across PCIe = 32 GB/s. Wrong placement = 28× slower communication.

How schedulers handle it

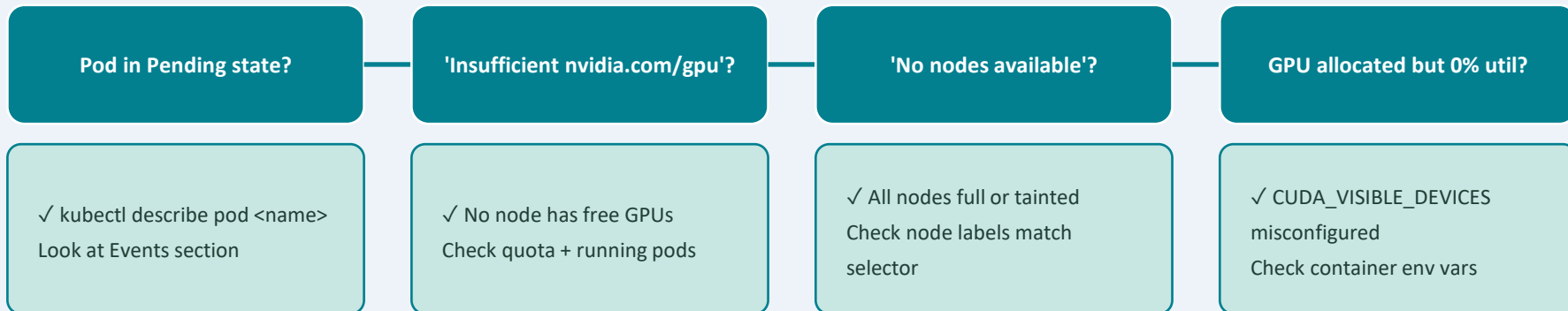
`nvidia-smi topo -m` shows the topology matrix. `CUDA_VISIBLE_DEVICES` controls which GPUs are visible. A topology-aware scheduler reads the matrix and prefers co-located GPUs for multi-GPU jobs.

```
# Show topology matrix
nvidia-smi topo -m
# GPU0 GPU1 GPU2 GPU3 ...
# GPU0  X   NV5  NV5  SYS
# GPU1  NV5  X   NV5  SYS
# ...
# NV5 = NVLink 5 (fast)
# SYS = cross-CPU socket via PCIe (slow)

# Kubernetes: topology manager + cpu manager
# kubelet --topology-manager-policy=best-effort
# --cpu-manager-policy=static

# NUMA-aware device plugin assigns GPUs
# in the same NUMA node as allocated CPUs
```

When Pods Stay Pending — A Diagnostic Flowchart



```
# Step 1: see why the pod is pending
kubectl describe pod <pod-name> -n <namespace>    # check Events

# Step 2: check node GPU capacity
kubectl get nodes -o custom-columns="NAME:.metadata.name,GPU:.status.allocatable.nvidia\.com/gpu"

# Step 3: check what's consuming the quota
kubectl describe resourcequota -n <namespace>

# Step 4: check device plugin is running on the node
kubectl get pods -n kube-system | grep nvidia-device-plugin
```

Why utilization is the number that matters

The waste patterns

Notebooks holding GPUs overnight at 0% · jobs sized 'GPU because habit' · data-loading bottlenecks leaving SMs starved · whole GPUs for tiny inference.

The levers

Idle reaping for interactive sessions · right-sizing via MIG · profiling input pipelines · utilization dashboards per team (visibility changes behaviour).

A modern 8-GPU node costs as much as several engineers' salaries per year. Idle GPUs are the most expensive idle thing you own.

Lab 2 — Watch a GPU actually work

```
# Terminal 1: a small training loop in your Week-2 container
docker run --rm --gpus all gpu-hello python3 busy_train.py

# Terminal 2: live device metrics, 1s resolution
$ nvidia-smi dmon -s um          # u=utilization  m=memory
# gpu    sm    mem    ...    fb(MB)
#    0    97    64      14210    <- healthy training
#    0     8     2      14210    <- input-starved: GPU waits on data

# Per-process accounting:
$ nvidia-smi pmon -c 5
```

- Experiment: shrink DataLoader workers to 0 and watch sm% collapse — that dip is money.

Week 2 recap — and the bridge to Week 3

- S1 — containers freeze userspace; the toolkit wires GPUs in at run time.
- S2 — lockfiles + the compatibility chain make environments deterministic; Jupyter needs port, volume, shm.
- S3 — a 3-level probe localizes GPU failures; CI rebuilds trust on every commit.
- S4 — schedulers turn GPUs into a metered, fair, measurable utility.
- **Week 3: Kubernetes for AI in depth — deployments, services, storage, device plugins, requests & limits.**